

dwes

CASO 1

O MISTÉRIO DO POD EM CRASHLOOPBACKOFF



LINUXTIPS

O CASO: O POD QUE NÃO QUERIA VIVER



Este é o rito de passagem para qualquer um que começa a usar o Kubernetes. Você passou horas criando o **deployment.yaml** perfeito para a sua aplicação. Você aplica o manifesto com **kubectl apply -f deployment.yaml** e o Kubernetes te responde com um amigável **deployment.apps/minha-app created**. Sucesso! Para confirmar, você vai listar os Pods, as unidades básicas onde seus containers vivem.

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
minha-app-6bcf89c9d-z8k2l	0/1	CrashLoopBackOff	5	2m

E aí está ele. O status que assombra os pesadelos de todo engenheiro: **CrashLoopBackOff**. Sim ou não, turma, que o coração até para por um segundo? O status **READY** está 0/1, o que significa que o container dentro do Pod não está pronto. E a coluna **RESTARTS** não para de aumentar. O Kubernetes está tentando, bravamente, iniciar seu container, mas ele falha, morre, e o Kubernetes, como um bom maestro, tenta de novo, em um loop infinito de falha e sofrimento.

*O que está acontecendo?
Por que o nosso Pod se recusa a viver?*

A INVESTIGAÇÃO: A AUTÓPSIA DO POD

Um **CrashLoopBackOff** não é o problema, é o sintoma.

O Kubernetes está nos dizendo:

"Eu estou fazendo o meu trabalho, que é tentar manter este Pod rodando, mas a aplicação dentro dele continua morrendo. O problema não é comigo, é com ele!".

Nossa missão é descobrir por que a aplicação está morrendo.

